CSE 3100

CODING PROBLEM MODIFICATION GUIDE

From a familiar template to an unfamiliar Exam 2 variation

Built for your two-hour, open-book exam: changes to process count, pipe directions, data types, loops, `exec()` commands, redirection, worker calculations, file reads, and output requirements.

Sources: CSE 3100 Part 2 Lectures 1–4, Lab 5, Lab 6, the Spring 2026 Exam 2 practice paper, and the previously prepared simplified FRQs 3–8 and harder solved examples.

Scope: The core APIs and patterns come from the source material. The modified prompts and code patches are newly designed practice variations; they are not predictions of the actual exam.

> Exam-policy reminder
> The supplied Spring 2026 practice paper permits certain local files but prohibits ChatGPT and other generative AI during the exam. Confirm the instructor permits prewritten template files; use this guide only within the rules.

How to use this guide: (1) read the question, (2) identify what CHANGED, (3) keep the original process/pipe skeleton where possible, (4) apply the matching patch, (5) compile and test.

Part I: Fast routing and safe modification rules | Part II: 29 change recipes | Part III: worked transformations and typing drills | Part IV: two-hour exam plan and submission checks

I. Start here: two-hour exam workflow

The practice exam itself states 110 minutes, but you report that your actual exam lasts two hours. The plan below uses 120 minutes. Adjust it if the instructor announces different timing or question weights.

Time window	Primary action	Why
0–15 min	MCQs: answer rules you know; mark uncertain ones	Secure straightforward points quickly
15–35 min	Four short FRQs: use concise model structures	Avoid losing easy five-point answers
35–105 min	Main coding problem: choose a template, modify, compile, test	Protect the most heavily weighted section
105–120 min	Review MCQs/FRQs; compile and verify submission	Catch missing branches, FDs, or output formatting

First 60 seconds of any coding FRQ

- Circle how many processes, who calls `fork()`, and whether each child runs a command or a calculation.
- Draw arrows for each data exchange. Each arrow = one direction. Assign a pipe to each directed connection for the simplest implementation.
- Mark data types: `char` (1 byte), `int` (`sizeof(int)`), `struct` (`sizeof(struct)`), or text until delimiter.
- Identify what is new relative to your closest template: number of stages, loop count, transformations, or destinations.
- Write the required output format before coding and check it later.

Find your starting template

Question wording	Base	What usually changes
Parent and Child calculate / reply	Two-way pipes	Sentinel, number of messages, calculation
Parent, C1, C2 exchange data	Three-process protocol	Pipe map, sends/reads, number of rounds
cmd1 | cmd2 | cmd3	Pipeline with dup2()	exec arguments and redirects
N children handle rows / tasks	Worker + tagged result	Computation, struct fields, worker count
Read text until comma / newline	File-reading loop	Delimiter and buffer limit
Child computes; Parent writes report	Pipe + output dup2()	Metrics and report target

> Golden rule
> Keep the scaffold when the communication directions remain unchanged. If directions or process count change, redraw the arrows and update pipe setup and `close()` calls before modifying calculations.

II. Universal rules before you patch anything

The shortest accurate API map

Call	What it does	Most common trap
fork()	Creates a child; returns 0 in Child	Does not share ordinary variable updates
waitpid(pid,&s,0)	Waits for/reaps specified child	Check WIFEXITED before WEXITSTATUS
execvp(file,argv)	Replaces current process; PATH search	argv must end with NULL
pipe(p)	Creates one-way byte channel	p[0] read; p[1] write
dup2(old,new)	Makes new FD refer to same open file description	Call before exec; close original if unneeded
open(path,flags,mode)	Opens file to get FD	O_CREAT needs a mode argument
read(fd,buf,n)	Reads up to n bytes	Return value can be smaller than n
write(fd,buf,n)	Writes up to n bytes	Handle partial writes if required

Minimal shared helpers for integer-based problems

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

// Exam-sized, one-int operations; error if incomplete.
static void put_int(int fd, int x) {
    if (write(fd, &x, sizeof x) != (ssize_t)sizeof x) {
        perror("write"); exit(1);
    }
}
static int take_int(int fd) {
    int x;
    if (read(fd, &x, sizeof x) != (ssize_t)sizeof x) {
        fprintf(stderr, "int read failed\n"); exit(1);
    }
    return x;
}
```

These compact helpers are convenient when reading or writing small fixed-size integers. The lecture notes emphasize that `read()` and `write()` return actual byte counts; production-grade code should loop on short transfers and retry interrupted calls.

III. Recipes 1–5: modifying `fork()` / `wait()` code

1. One child becomes two independent children

Keep: parent-child branching and error checks. Change: add a second `fork()` only in the original parent. Create both children before waiting if they should run concurrently.

```
pid_t c1 = fork();
if (c1 < 0) { perror("fork"); return 1; }
if (c1 == 0) { /* task A */ _exit(0); }

pid_t c2 = fork();
if (c2 < 0) { perror("fork"); waitpid(c1,NULL,0); return 1; }
if (c2 == 0) { /* task B */ _exit(0); }

waitpid(c1, NULL, 0);
waitpid(c2, NULL, 0);
```

Why: The first child exits before it could reach the second `fork()`. Otherwise you may create unintended grandchildren.

2. Concurrent commands become sequential commands

Move the first `waitpid()` immediately before the second `fork()`. The first child must finish before the second begins.

```
pid_t c1 = fork();
if (c1 == 0) { execlp("date","date",(char*)NULL); _exit(127); }
waitpid(c1, NULL, 0);   // new ordering constraint
pid_t c2 = fork();
if (c2 == 0) { execlp("ls","ls",(char*)NULL); _exit(127); }
waitpid(c2, NULL, 0);
```

Add `fork()` failure checks in the completed program; this patch emphasizes order.

3. N children, each performing one task

```
pid_t pid[N];
for (int i = 0; i < N; ++i) {
    pid[i] = fork();
    if (pid[i] < 0) { perror("fork"); exit(1); }
    if (pid[i] == 0) {
        /* task for index i */
        _exit(0);     // REQUIRED: child cannot fork next time
    }
}
for (int i = 0; i < N; ++i)
    waitpid(pid[i], NULL, 0);
```

Change `N`, the work assignment, and any result array sizes together. Without child exit, successive forks create additional processes.

4. Print order must be guaranteed

A `fork()` does not guarantee whether parent or child executes first. If question requires Child line before Parent line, wait for Child before printing Parent line.

```
// Parent branch:
waitpid(child, NULL, 0);
printf("Parent after child\n");
```

5. Report exit status (normal or signal termination)

```
int status;
if (waitpid(pid, &status, 0) == -1) perror("waitpid");
else if (WIFEXITED(status))
    printf("exit=%d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status))
    printf("signal=%d\n", WTERMSIG(status));
```

Do not use `WEXITSTATUS(status)` unless `WIFEXITED(status)` is true. For the practice exam, `return 257` is observed as 1 on typical POSIX systems.

IV. Recipes 6–13: pipe protocol modifications

6. One-way Child -> Parent becomes two-way Parent <-> Child

Keep: one child fork. Change: create two pipes before fork and give each side one read end and one write end.

```
int p2c[2], c2p[2];
pipe(p2c); pipe(c2p);
pid_t pid = fork();
if (pid == 0) {
    close(p2c[1]); close(c2p[0]);
    int x = take_int(p2c[0]);      // child receives
    put_int(c2p[1], x * 2);        // child responds
    close(p2c[0]); close(c2p[1]);
    _exit(0);
}
close(p2c[0]); close(c2p[1]);
put_int(p2c[1], 10);               // parent requests
int answer = take_int(c2p[0]);    // parent receives
printf("%d\n", answer);           // 20
close(p2c[1]); close(c2p[0]);
waitpid(pid, NULL, 0);
```

7. One request becomes many requests

Keep the two pipes. Wrap each request and response in a loop on both sides, and define how the child learns that requests are finished.

```
// Child:
while (1) {
    int x = take_int(p2c[0]);
    if (x == -1) break;             // sentinel
    put_int(c2p[1], x * x);
}
// Parent (uses pipes already created/open):
int inputs[] = {2, 3, 4};
for (int i = 0; i < 3; ++i) {
    put_int(p2c[1], inputs[i]);
    printf("%d\n", take_int(c2p[0]));
}
put_int(p2c[1], -1);               // tell child to stop
```

Avoid choosing a sentinel that is also a legitimate input unless the protocol specifically reserves it.

8. `char` messages become binary `int` messages

```
// OLD: one character
char x;
read(p[0], &x, 1);
write(p[1], &x, 1);

// NEW: one integer
int x;
read(p[0], &x, sizeof x);
write(p[1], &x, sizeof x);
```

Also change `%c` to `%d`, character arithmetic to integer arithmetic, and initial argument parsing (`strtol()` instead of `argv[1][0]`). Check read/write return values in final code.

9. A fixed integer response becomes a newline-terminated message

```
// Child sends final text AFTER last binary response:
char msg[80];
int n = snprintf(msg, sizeof msg, "Done: %d items\n", count);
if (n < 0 || n >= (int)sizeof msg) _exit(1);
write(c2p[1], msg, (size_t)n);

// Parent expects ASCII AFTER reading final integer:
char ch;
while (read(c2p[0], &ch, 1) == 1) {
    putchar(ch);
    if (ch == '\n') break;
}
```

Pipe data is a stream of bytes. The parent must know when the protocol switches from binary integers to ASCII text.

10. Parent -> C1 -> C2 -> Parent ring

Use three pipes and keep exactly one input and output per process. This is a simpler variation of the six-exchange practice-exam problem.

Process	Read	Write
Parent	c2p[0]	p2c1[1]
Child 1	p2c1[0]	c1c2[1]
Child 2	c1c2[0]	c2p[1]

```
// Parent round:
put_int(p2c1[1], x);
x = take_int(c2p[0]);

// Child 1 round:
int v = take_int(p2c1[0]);
put_int(c1c2[1], v * 2);

// Child 2 round:
int v = take_int(c1c2[0]);
put_int(c2p[1], v + 5);
```

Input 3 -> Child 1 sends 6 -> Child 2 sends 11 -> Parent receives 11. No fourth pipe needed unless the parent must send a separate reply to C2.

11. Six-exchange protocol: modify operations, not connections

The actual practice exam is one round: Parent->C1->C2->Parent->C2->C1->Parent. The simplified FRQ 4 you practiced expanded that same route to three rounds. If the question changes only calculations, all six pipes and closes remain the same.

Step / Pipe	Who changes it?	Change in code
1 / p[0]	Parent sends input	send_char(p[0][1], x)
2 / p[1]	Child 1 transforms	x += 2; send_char(p[1][1], x)
3 / p[2]	Child 2 transforms	x += 3; send_char(p[2][1], x)
4 / p[3]	Parent transforms	x -= 1; send_char(p[3][1], x)
5 / p[4]	Child 2 transforms	x *= 2; send_char(p[4][1], x)
6 / p[5]	Child 1 transforms	x += 1; send_char(p[5][1], x)

For numeric inputs, starting at 2 gives 2 -> 4 -> 7 -> 6 -> 12 -> 13. To repeat, use a loop around all operations in every process with matching iteration counts; the Parent reuses 13 for the next round.

12. Add a feedback/acknowledgment stage

If Parent must send an acknowledgment to Child 2 after receiving the result, add a new Parent->C2 pipe; update both close tables and add matching write/read in the same order.

```
int p2c2[2]; pipe(p2c2);   // BEFORE either fork

// Parent keeps p2c2[1]; closes p2c2[0].
// After getting result from c2p:
int ack = (result % 2 == 0);
put_int(p2c2[1], ack);

// Child 2 keeps p2c2[0]; closes p2c2[1].
// After sending result to Parent:
int ack = take_int(p2c2[0]);
if (ack) approvals++;
```

A new read with no matching write may hang. Every sender, receiver, and close list must be updated consistently.

13. Reverse an arrow / change the communication order

Do not keep the original pipe table. A reversed direction requires the new receiver to keep the read end and the new sender to keep the write end. Redraw arrows first; then rewrite closes and read/write order, not just the helper call.

```
// BEFORE: C1 -> C2 through p[1]
// C1: write(p[1][1],...); C2: read(p[1][0],...);

// AFTER: C2 -> C1 through p[1]
// C2: write(p[1][1],...); C1: read(p[1][0],...);
// IMPORTANT: swap which ends C1 and C2 KEEP.
```

V. Recipes 14–19: `exec()` and command pipelines

14. Fixed command -> variable command-line arguments

For a known number of arguments, `execlp()` is short. For a variable number, use `execvp()` and a NULL-terminated array.

```
// Fixed command:
execlp("wc", "wc", "-l", "input.txt", (char*)NULL);

// Variable command from argv[1..argc-1]:
execvp(argv[1], &argv[1]);
perror("execvp");
_exit(127);
```

`execl()` / `execv()` take an explicit executable path; `execlp()` / `execvp()` search `PATH`. A successful `exec()` does not return.

15. Two commands -> three commands

For the simple line A | B, use one pipe. For A | B | C, use two pipes and three separately forked children. Keep the three process branches explicit, as in your simplified FRQ 5.

Stage	stdin / FD 0	stdout / FD 1
A (first)	Original or file	a[1] via dup2

Stage	stdin / FD 0	stdout / FD 1
B (middle)	a[0] via dup2	b[1] via dup2
C (last)	b[0] via dup2	Original or file

```
// Child A: output to pipe a
if (dup2(a[1], STDOUT_FILENO) == -1) _exit(1);
// Child B: input from a, output to b
if (dup2(a[0], STDIN_FILENO) == -1) _exit(1);
if (dup2(b[1], STDOUT_FILENO) == -1) _exit(1);
// Child C: input from b
if (dup2(b[0], STDIN_FILENO) == -1) _exit(1);
// In EACH child: close all a[0],a[1],b[0],b[1]
// after duplicating; then exec its assigned command.
```

16. Add stdin redirection to first command

```
int in = open(argv[1], O_RDONLY);
if (in == -1) { perror("open"); _exit(1); }
if (dup2(in, STDIN_FILENO) == -1) _exit(1);
close(in);
// then dup2(pipe_write, STDOUT_FILENO) and exec
```

Use this when the prompt shows `command < input.txt | command2`. Do not also supply the filename as a positional argument unless that executable requires it.

17. Add output redirection to last command

```
int out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (out == -1) { perror("open"); _exit(1); }
if (dup2(out, STDOUT_FILENO) == -1) _exit(1);
close(out);
// then exec final command
```

For `>> report.txt`, use `O_APPEND` in place of `O_TRUNC`. The `O_CREAT` flag requires the mode argument.

18. Redirect stderr but preserve stdout pipeline

```
int err = open("errors.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
if (err == -1) { perror("open"); _exit(1); }
dup2(err, STDERR_FILENO);  // affects FD 2, NOT FD 1
close(err);
// stdout may still go into pipe using dup2(pipe[1], 1)
```

`stderr` remains FD 2; `stdout` remains FD 1. The first command may send regular results down the pipe while diagnostics go to a separate file.

19. Add a third-party executable after `exec`

You cannot perform more work in the original child *after a successful `exec()`*. If the question asks for a second program to run concurrently, create another child. If it asks for a separate stage in a pipeline, create another pipe and child. If sequential, wait and then fork another child.

VI. Recipes 20–24: parallel workers and file I/O

20. Row sums -> row maxima / minima / averages

```
// Child has row index i, 3 columns.
int sum = 0, mx = a[i][0], mn = a[i][0];
for (int j = 0; j < 3; ++j) {
    sum += a[i][j];
    if (a[i][j] > mx) mx = a[i][j];
    if (a[i][j] < mn) mn = a[i][j];
}
double average = sum / 3.0;
```

If you send the average in a struct, use a `double` field and print with `%f` or `%.2f`. If you send only integer results, keep the existing struct and replace the computation.

21. One returned integer -> tagged structure

```
typedef struct { int row; int sum; int maximum; } Result;
// Child after computing:
Result r = {i, sum, mx};
write(p[1], &r, sizeof r);
// Parent receiving one complete record:
Result r;
read(p[0], &r, sizeof r);
results[r.row] = r;
```

Multiple writers can finish in any order. Tag each result with the row/task ID. Small one-write records (<= PIPE_BUF) do not interleave with other writers, but read can still be short; use a loop when robustness is required.

22. Three row workers -> N workers

Change the matrix dimensions, `pid_t ids[N]`, result array, all loop bounds, and assigned tasks together. Never assume the first result read belongs to worker 0. Always terminate a worker before the outer fork loop repeats.

23. Read until comma -> newline / colon / period

```
char buffer[100], ch;
int i = 0;
ssize_t n;
while ((n = read(fd, &ch, 1)) == 1) {
    if (ch == ':') break;  // CHANGE delimiter here
    if (i < (int)sizeof(buffer) - 1)
        buffer[i++] = ch;
}
if (n == -1) perror("read");
buffer[i] = '\0';     // read() DOES NOT add this
```

To include the delimiter, store it first (if space remains), then test for it. To process arbitrarily long inputs without truncation, grow a buffer or stream a result instead of storing the entire string.

24. Read until EOF -> count bytes, commas, and lines

```
int bytes = 0, commas = 0, lines = 0;
char ch, last = '\0';
ssize_t n;
while ((n = read(fd, &ch, 1)) == 1) {
    ++bytes;
    if (ch == ',') ++commas;
    if (ch == '\n') ++lines;
    last = ch;
}
if (n == -1) { perror("read"); /* handle */ }
if (bytes > 0 && last != '\n') ++lines;
```

This counts logical lines: each newline plus a nonempty last line without a newline. It differs from `wc -l`, which counts newline characters.

VII. Recipes 25–29: fixes for common exam twists

25. File offset changes via `lseek()`

```
lseek(fd, 0, SEEK_SET);   // beginning
lseek(fd, 2, SEEK_CUR);   // forward 2 from current
lseek(fd, 0, SEEK_END);   // end of seekable file
```

Duplicated descriptors (`dup`, `dup2`) and descriptors inherited across fork share an underlying open-file offset. A separate `open()` call creates a separate open-file description. `lseek()` generally does not apply to pipes.

26. Required output order changes

Process scheduling is nondeterministic. If an output must occur after a child has completed, use `waitpid()`. If it must occur after a *specific message*, print immediately after receiving that message. A synchronization point is not equivalent to just moving the print line between independent processes.

27. Program hangs: how to debug

- Check that each `read()` has a possible writer and vice versa.
- Check every process closed unused write ends so readers can receive EOF.
- Check Parent does not `waitpid()` before it finishes sending data required by Child.
- Check request/response order: if both processes begin by `read()`, they both block.
- After `dup2()` in pipeline, close the original pipe ends in every child and parent.

28. Return value check and partial transfer handling

```
// A robust helper for reading n bytes (a pipe byte stream):
void read_exact(int fd, void *buf, size_t n) {
    char *p = buf;
    while (n > 0) {
        ssize_t k = read(fd, p, n);
        if (k < 0 && errno == EINTR) continue;
        if (k < 0) { perror("read"); exit(1); }
        if (k == 0) { fprintf(stderr, "Unexpected EOF\n"); exit(1); }
        p += k;
        n -= (size_t)k;
    }
}
void write_exact(int fd, const void *buf, size_t n) {
    const char *p = buf;
    while (n > 0) {
        ssize_t k = write(fd, p, n);
        if (k < 0 && errno == EINTR) continue;
        if (k < 0) { perror("write"); exit(1); }
        if (k == 0) { fprintf(stderr, "No write progress\n"); exit(1); }
        p += k;
        n -= (size_t)k;
    }
}
// Requires #include <errno.h>
```

Use the short helpers for exam-sized simple cases if your assumptions permit; use exact-read / exact-write loops when the problem demands robust handling. `write()` atomicity for <= PIPE_BUF does not mean one `read()` necessarily returns a full record.

29. A child writes data but Parent sees an unchanged variable

Normal variables are copied at `fork()`. Use a pipe to transmit the result. Simply calling `sleep()` or `waitpid()` does not make Child changes appear in Parent memory.

VIII. Fully worked exam-style transformations

Example A: practice exam route -> numeric transforms and two rounds

Prompt variation: Preserve the same six directions as the real practice exam. Send `int` instead of `char`; C1 doubles on step 2 and adds 1 on step 6; C2 adds 3 on step 3 and doubles on step 5; Parent subtracts 1 on step 4; do two rounds starting at 2.

Keep: six pipes, the Parent/C1/C2 branching, the exact close tables, and operation order. Change: char helpers -> integer helpers; calculations; loop count in all three processes; integer input parsing and output formats.

Step	First round	Second round
P -> C1	2	13
C1 -> C2 (x2)	4	26
C2 -> P (+3)	7	29
P -> C2 (-1)	6	28
C2 -> C1 (x2)	12	56

Step	First round	Second round
C1 -> P (+1)	13	57

```
// Child 1 repeated round (after closing unused FDs):
for (int i = 0; i < 2; ++i) {
    int x = take_int(p[0][0]);
    put_int(p[1][1], x * 2);
    x = take_int(p[4][0]);
    put_int(p[5][1], x + 1);
}

// Child 2 repeated round:
for (int i = 0; i < 2; ++i) {
    int x = take_int(p[1][0]);
    put_int(p[2][1], x + 3);
    x = take_int(p[3][0]);
    put_int(p[4][1], x * 2);
}

// Parent repeated round:
int x = 2;
for (int i = 0; i < 2; ++i) {
    put_int(p[0][1], x);
    x = take_int(p[2][0]);
    put_int(p[3][1], x - 1);
    x = take_int(p[5][0]);
    printf("round %d: %d\n", i+1, x);
}
```

Expected Parent rounds: `round 1: 13` then `round 2: 57`. Common errors: changing only Parent loop; forgetting sizeof(int); leaving `%c`; or using `argv[1][0]` for a numeric argument.

Example B: `cat | tr | sort` -> `grep -i error < in | wc -l > count`

Keep: create pipes before forks; `dup2()` before `exec()`; close raw pipe FDs. Change: use two commands (one pipe), add input redirection for first child, output redirection for second, and replace `execlp()` calls.

```
// Child 1, grep (after fork):
int in = open(argv[1], O_RDONLY);
if (in == -1) _exit(1);
dup2(in, STDIN_FILENO);
dup2(p[1], STDOUT_FILENO);
close(in); close(p[0]); close(p[1]);
execlp("grep","grep","-i","error",(char*)NULL);
perror("exec grep"); _exit(127);

// Child 2, wc:
int out = open(argv[2], O_WRONLY|O_CREAT|O_TRUNC, 0644);
if (out == -1) _exit(1);
dup2(p[0], STDIN_FILENO);
dup2(out, STDOUT_FILENO);
close(out); close(p[0]); close(p[1]);
execlp("wc","wc","-l",(char*)NULL);
perror("exec wc"); _exit(127);
```

Critical: The parent must close `p[0]` and `p[1]` and wait for both children. For a 3-command variation, add another pipe and child instead of incorrectly connecting `grep` directly to `sort` and `wc`.

Example C: row sums -> per-row maximum and average

Keep: one parent-readable pipe, fork N children, tagged result, ordered storage. Change: calculation and `Result` fields.

```
typedef struct {
    int row;
    int maximum;
    double average;
} Result;

// In worker for row i with 3 columns:
int sum = 0, mx = a[i][0];
for (int j = 0; j < 3; ++j) {
    sum += a[i][j];
    if (a[i][j] > mx) mx = a[i][j];
}
Result r = {i, mx, sum / 3.0};
write(p[1], &r, sizeof r);

// Parent stores by r.row, prints after all reads:
printf("row %d: max %d avg %.2f\n",
       r.row, r.maximum, r.average);
```

If different workers finish in different orders, always index `results[r.row]`, not `results[i]` where `i` is the arrival index.

Example D: file before comma -> count all commas and logical lines

Keep: `open(O_RDONLY)` and byte-by-byte loop. Change: remove the `break`, replace string storage with counters, and handle a final line without newline.

```
int commas=0, lines=0, bytes=0; char c, last='\0';
ssize_t n;
while ((n=read(fd,&c,1))==1) {
    ++bytes;
    if (c==',') ++commas;
    if (c=='\n') ++lines;
    last=c;
}
if (bytes>0 && last!='\n') ++lines;
printf("commas %d lines %d\n",commas,lines);
```

For contents `A,B C,D` with no final newline: commas=2, logical lines=2. `wc -l` would count just one newline character.

IX. Typing drills: modify without rewriting everything

Use the original solved template, then apply one of these changes. Solutions below specify what must change, which is the skill you need for an open-book coding problem.

Drill	Change demanded	Minimum patch
1	6 exchanges -> one round	Loop 3 -> 1 in every process
2	C1->C2 direction reverses	Redraw arrows; flip owners of p[1] read/write; rewrite order

Drill	Change demanded	Minimum patch
3	Child loops until -1	Child tests sentinel; Parent sends -1 after requests
4	Add a third pipeline command	New pipe b; third child; middle dup2 twice
5	Worker row sum -> maximum	Initialize from a[i][0]; update if >
6	Send float average instead of int sum	Result struct has double; compute /3.0; print %.2f
7	Pipe program sometimes deadlocks	Close all extra writer FDs; match send/read order
8	Send stderr to diagnostics file	open errors; dup2(err,2); close err before exec
9	Read to colon instead of comma	Use if (c == ':') (single-quoted char)
10	Order of children must be deterministic	waitpid at required point or handshake

X. Partial-credit-safe code construction order

There is no published partial-credit rubric in the supplied practice paper. Nevertheless, a clear, compiling skeleton plus correct process structure is easier for a grader to evaluate than a long, noncompiling attempt.

- First write the required headers, `main(int argc,char **argv)`, and argument validation.
- Declare and create the necessary pipes *before* any fork.
- Create each child in a separate branch and place `_exit(0)` at the end of its work.
- Fill each process's descriptor-close section from the pipe direction table.
- Write send/receive operations in the exact listed protocol order, then calculations.
- Add output labels and check expected sample characters/numbers.
- Close remaining descriptors and `waitpid()` in Parent.
- Compile with `gcc -Wall -Wextra -std=c11 solution.c -o solution` and test both given and edge inputs.

XI. One-page last-minute reference

If exam says...	Think...
New child or multiple tasks	fork() (not exec()), with error checks
Run executable	execvp() for argv array / variable arguments
One-way exchange	One pipe; [0] reads, [1] writes
Two-way conversation	Two distinct pipes with opposite directions
Parent, C1, C2 many steps	Draw arrows and assign directed pipes
A | B	One pipe; A stdout -> write; B stdin <- read
A | B | C	Two pipes; middle command stdin + stdout redirected
< file	dup2(in, STDIN_FILENO) before exec()

If exam says...	Think...
> file	dup2(out, STDOUT_FILENO)
2> file	dup2(err, STDERR_FILENO)
Children finish out of order	Attach ID to result and store results[r.id]
Line/word/char counts	read() loop; use correct delimiter rule
File appears unchanged in Parent	Child memory separate; communicate via pipe
read() blocks	Check messages, close writers, EOF, protocol order

Important exam-source distinctions: (1) the practice paper is a single-round character protocol, not the three-round extension; (2) successful `exec()` replaces the program; (3) a fork child receives return value 0 but does not have PID 0; (4) an `open()` with `O_CREAT` requires a mode; (5) pipe writes <= PIPE_BUF are atomic, yet reads still may be short.

Source map

- Lecture 1: fork behavior, independent memory, process count, concurrent scheduling, wait and zombies.
- Lecture 2: exec family, child error paths, file APIs, flags, descriptors inherited by fork/exec.
- Lecture 3: stdin/stdout/stderr, open/dup/dup2, output redirection, child-only redirects.
- Lecture 4: pipes, two-way communication, pipelines, descriptor cleanup, read/write semantics, parallel row sums.
- Lab 5 / Lab 6: exec command launching and parent-child guessing protocols.
- Spring 2026 Exam 2 practice: 10 MCQs, four 5-point FRQs, one 50-point six-exchange coding problem; published 110-minute sample, whereas your reported actual exam limit is 120 minutes.